SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1101

COURSE OUTCOME COVERED

CO1: *Apply object-oriented concepts and software development life cycle models to design structured and modular software systems.* *(BL3)*

Assessment Tool Weightage: 40%

Dr M. Ramamoorthy

QUESTIONS: **Total Marks:40**

Annexure A

TECHNICAL PRESENTATION

Code of Conduct:

I Mathusri P-192311363 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *Mathusri P*

Annexure A

TECHNICAL PRESENTATION

1) Object-Oriented Design in Smart Healthcare Systems

Introduction

A Smart Healthcare System is a software application designed to manage healthcare services efficiently by maintaining information about patients, doctors, appointments, prescriptions, laboratory reports, and medical records. As the number of users and medical services increases, the software must be secure, reliable, scalable, and easy to maintain. Object-Oriented Design (OOD) provides an effective approach for developing such systems by organizing software into classes and objects. It improves software quality by promoting modularity, code reuse, flexibility, and easier maintenance.

Application of Object-Oriented Concepts

1. Class

A class is a blueprint used to create objects. It defines the properties (attributes) and functions (methods) of an object. In a Smart Healthcare System, different classes are created to represent different entities.

Examples of classes include:

- Patient
- Doctor
- Appointment
- Medical Record
- Prescription
- Laboratory Report

Each class contains relevant information and operations associated with that particular entity.

2. Object

An object is an instance of a class. Each object contains its own data and performs specific functions.

Examples:

- Patient object – Rahul Kumar
- Doctor object – Dr. Priya
- Appointment object – Appointment No. 105
- Medical Record object – Patient History

Objects interact with one another to perform healthcare-related operations such as booking appointments, generating prescriptions, and updating patient records.

3. Encapsulation

Encapsulation combines data and methods into a single unit and protects sensitive information by restricting direct access. Data can only be accessed through authorized methods.

In the healthcare system, confidential information such as:

- Patient medical history
- Blood group
- Laboratory reports
- Personal details

Advantages:

- Protects confidential patient information.
- Prevents unauthorized modification of records.
- Improves data security.
- Maintains data integrity.

4. Inheritance

Inheritance allows one class to acquire the properties and methods of another class. This reduces duplicate code and promotes reusability.

For example, a common class called **Person** can contain common details such as:

- Name
- Age
- Address
- Contact Number

The classes **Doctor**, **Patient**, and **Administrator** can inherit these common properties while adding their own specific features.

Advantages:

- Reduces code duplication.
- Improves software reusability.
- Simplifies future development.
- Makes maintenance easier.

5. Polymorphism

Polymorphism allows one method to perform different tasks depending on the object that calls it.

For example, the method **generateReport()** may produce:

- Patient Report
- Laboratory Report
- Billing Report
- Appointment Report

Similarly, the method **calculateBill()** can calculate different charges for:

- Outpatient
- Inpatient
- Emergency Patient

This increases flexibility and makes the software easier to extend.

Analysis of Object-Oriented Design

Scalability

Object-Oriented Design allows new modules such as Pharmacy Management, Telemedicine, or Online Consultation to be added without affecting existing modules. This enables the system to grow as hospital requirements increase.

Security

Encapsulation ensures that sensitive patient information is accessible only to authorized users such as doctors and administrators. This improves patient privacy and reduces the risk of data leakage.

Maintainability
Since the software is divided into independent classes and modules, developers can modify one part of the system without affecting the remaining components. This reduces maintenance cost and improves software reliability.

Code Reusability
Inheritance enables developers to reuse existing classes instead of writing the same code repeatedly. This saves development time and improves software quality.

Advantages of Object-Oriented Design
- Improves software organization.
- Increases code reusability.
- Enhances system security.
- Supports scalability.
- Simplifies maintenance.
- Reduces development time.
- Makes debugging easier.
- Improves overall software quality.

Conclusion
Object-Oriented Design plays a vital role in the development of Smart Healthcare Systems. Concepts such as class, object, encapsulation, inheritance, and polymorphism help build secure, flexible, scalable, and maintainable software. These concepts enable healthcare organizations to manage patient information efficiently while providing reliable and high-quality healthcare services.

---

2)Applying Encapsulation and Abstraction in Online Banking

Introduction
An Online Banking System enables customers to perform banking operations such as account management, fund transfer, balance enquiry, bill payment, and transaction tracking through the internet. Since banking applications deal with sensitive financial information, they require strong security and a simple user interface. Two important object-oriented concepts that support these requirements are **Encapsulation** and **Abstraction**. These concepts improve software security, reduce complexity, and provide a better user experience.

Encapsulation
Encapsulation is the process of combining data and methods into a single class while preventing direct access to sensitive information. Data can only be accessed using authorized methods.
In an Online Banking System, the following information is protected using encapsulation:
- Account Number
- Customer Name
- Account Balance
- ATM PIN
- Transaction History
- Debit and Credit Details

Users cannot directly modify these values. Instead, operations such as deposit, withdrawal, and balance enquiry are performed through secure methods.

Advantages of Encapsulation

- Protects confidential customer information.
- Prevents unauthorized access.
- Maintains data integrity.
- Improves software security.
- Simplifies maintenance.

Abstraction

Abstraction hides internal implementation details and provides only essential features to users. Customers interact only with simple banking services while complex internal processing remains hidden.

Users can access services such as:

- Login
- Fund Transfer
- Balance Enquiry
- Mini Statement
- Loan Application

The system hides processes such as:

- Database operations
- Password encryption
- Transaction verification
- Server communication
- Security validation

Advantages of Abstraction

- Reduces software complexity.
- Improves user experience.
- Simplifies application usage.
- Supports modular programming.
- Makes software easier to maintain.

Comparison between Encapsulation and Abstraction

Encapsulation

- Protects data.
- Restricts direct access.
- Focuses on data security.
- Uses private variables and public methods.

Abstraction

- Hides implementation details.
- Displays only essential features.
- Focuses on simplicity.
- Uses abstract classes and interfaces.

Analysis

Encapsulation protects customer accounts from unauthorized access and prevents accidental data modification. Abstraction simplifies banking operations by hiding technical implementation details. Together, they improve system security, reliability, and customer satisfaction while making the software easier to develop and maintain.

Conclusion
Encapsulation and Abstraction are essential principles in Online Banking Systems. Encapsulation protects sensitive customer information, while abstraction simplifies user interaction. Together, these concepts provide secure, reliable, user-friendly, and maintainable banking software.

3) Modeling Object Relationships in a University Management System

Introduction
A University Management System is a software application used to manage various academic and administrative activities of a university. It stores and processes information related to students, faculty members, departments, courses, examinations, attendance, and results. To develop such a system effectively, Object-Oriented Analysis and Design (OOAD) uses different object relationships. The three important object relationships are **Association, Aggregation, and Composition**. These relationships help represent real-world entities accurately and improve software flexibility, reusability, and maintainability.

Object Relationships in a University Management System
1. Association
Association is a relationship where two or more objects communicate with each other while remaining independent. One object can exist without the other.

For example:
- A **Student** enrolls in one or more **Courses**.
- A **Faculty Member** teaches multiple **Courses**.
- A **Department** offers different **Courses**.

In this relationship, both objects have their own independent existence. If a student leaves the university, the course still exists. Similarly, if a course is discontinued, the student still exists.

Advantages of Association
- Represents real-world interactions effectively.
- Supports communication between different objects.
- Provides flexibility in system design.
- Makes the software easier to expand.
- Reduces dependency among objects.

2. Aggregation
Aggregation is a "Has-A" relationship where one object contains another object, but both objects can exist independently.

For example:
- A **Department** has many **Faculty Members**.
- A **University** has many **Departments**.
- A **Course** has multiple **Students**.

If one department is removed, faculty members can still exist and may be assigned to another department.

Advantages of Aggregation

- Promotes loose coupling between objects.
- Improves modularity.
- Makes software more flexible.
- Simplifies code maintenance.
- Supports object reusability.

3. Composition

Composition is a strong "Part-Of" relationship where the child object cannot exist without the parent object.

For example:

- A **Student** has a **Student ID Card**.
- A **Course** contains **Course Materials**.
- An **Examination** contains **Answer Sheets**.

If the parent object is deleted, the child object is also deleted because it depends completely on the parent.

Advantages of Composition

- Ensures strong ownership.
- Improves data consistency.
- Prevents unnecessary objects.
- Simplifies object lifecycle management.
- Enhances system reliability.

\

Importance of Proper Relationship Modeling

1. Improves Flexibility

Proper object relationships allow the system to adapt to future changes easily. New courses, departments, or faculty members can be added without affecting existing modules.

2. Enhances Reusability

Reusable classes reduce duplicate coding and improve software quality. Existing objects can be reused in different modules, reducing development time.

3. Improves Maintainability

When software is divided into independent objects and relationships, developers can update one module without affecting the rest of the system. This reduces maintenance effort and debugging time.

4. Better Software Organization

Object relationships organize data logically, making the system easier to understand, develop, and maintain.

5. Reduces Complexity

Relationship modeling simplifies complex university operations by dividing them into manageable objects with clear responsibilities.

Applications in a University Management System
Object relationships are used in:

- Student Registration System
- Course Management
- Faculty Management
- Examination Management
- Attendance System
- Library Management
- Result Processing
- Fee Management

Benefits of Object Relationship Modeling

- Represents real-world relationships accurately.
- Improves software flexibility.
- Enhances code reusability.
- Reduces development time.
- Simplifies debugging.
- Improves maintainability.
- Supports future system expansion.
- Produces organized and modular software.

Conclusion
Association, Aggregation, and Composition are essential object relationships used in Object-Oriented Analysis and Design. They help represent the relationships among students, faculty, departments, and courses effectively. Proper relationship modeling improves flexibility, reusability, maintainability, and scalability, making the University Management System more efficient, reliable, and easier to manage.

---

4) Importance of SDLC in E-Commerce Development

Introduction
An E-Commerce Platform is an online application that allows customers to browse products, place orders, make online payments, and track deliveries. Developing such a system requires careful planning and execution to ensure reliability, security, and customer satisfaction. The **Software Development Life Cycle (SDLC)** provides a systematic process for developing high-quality software. Each phase of SDLC plays an important role in delivering a successful and user-friendly E-Commerce application.

Phases of Software Development Life Cycle (SDLC)
1. Requirement Analysis
Requirement Analysis is the first phase of SDLC. In this phase, developers collect and analyze the requirements provided by customers and stakeholders.

Activities include:

- Understanding customer needs.
- Identifying system requirements.
- Defining project objectives.
- Preparing requirement documents.

Importance

- Reduces misunderstandings.
- Defines project scope clearly.
- Helps estimate project cost and time.
- Ensures the software meets customer expectations.

2. System Design

After gathering requirements, the system architecture and design are prepared.

Activities include:

- Designing system architecture.
- Creating database structure.
- Designing user interface.
- Planning software modules.
-

Importance

- Provides a clear development roadmap.
- Improves software performance.
- Reduces future design issues.
- Supports efficient coding.

3. Coding (Implementation)

In this phase, developers convert the design into a working software application using programming languages.

Activities include:

- Writing source code.
- Developing application modules.
- Integrating payment gateways.
- Connecting databases.

Importance

- Produces functional software.
- Implements customer requirements.
- Ensures software functionality.
- Forms the core of application development.

4. Testing

Testing ensures that the developed software works correctly and satisfies customer requirements.

Types of testing include:

- Unit Testing
- Integration Testing
- System Testing
- User Acceptance Testing

Importance

- Identifies software defects.
- Improves software reliability.
- Ensures security.

- Enhances software quality.
- Reduces system failures after deployment.

5. Deployment
Deployment is the process of installing and releasing the software for customer use.

Activities include:
- Installing the application.
- Configuring servers.
- Publishing the website.
- Monitoring initial performance.

Importance
- Makes software available to users.
- Ensures successful software launch.
- Provides real-time customer access.
- Supports business operations.

6. Maintenance
Maintenance is the final phase of SDLC and continues throughout the software's lifetime.

Activities include:
- Fixing software bugs.
- Updating security features.
- Improving application performance.
- Adding new functionalities.

Importance
- Keeps software up to date.
- Improves customer satisfaction.
- Enhances software security.
- Extends software lifespan.
- Supports changing business requirements.

Contribution of SDLC to Software Quality
1. Improves Software Quality
Each SDLC phase ensures that errors are identified early and corrected before the software is delivered.

2. Enhances Reliability
Systematic development and testing improve software stability and reduce failures during operation.

3. Increases Customer Satisfaction
By following customer requirements throughout development, the final product meets user expectations and provides a better experience.

4. Reduces Development Risk
Proper planning, design, and testing reduce project risks, delays, and unexpected failures.

5. Simplifies Maintenance
A well-designed software system is easier to maintain, upgrade, and expand as business requirements change.

Benefits of SDLC in E-Commerce Development

- Produces high-quality software.
- Improves system security.
- Reduces development cost.
- Enhances software reliability.
- Supports timely project completion.
- Increases customer satisfaction.
- Simplifies future maintenance.
- Enables continuous system improvement.

Conclusion
The Software Development Life Cycle (SDLC) is essential for developing a successful E-Commerce Platform. Each phase—from Requirement Analysis to Maintenance—contributes to software quality, reliability, security, and customer satisfaction. Following SDLC helps developers build scalable, maintainable, and user-friendly applications that meet business goals and provide a seamless online shopping experience.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness